



ИНСТИТУТ ЗА МАТЕМАТИКУ И ИНФОРМАТИКУ
ПРИРОДНО-МАТЕМАТИЧКИ ФАКУЛТЕТ
УНИВЕРЗИТЕТ У КРАГУЈЕВЦУ

ЗАВРШНИ РАД

**ГЕНЕРИСАЊЕ И ВАЛИДАЦИЈА
ДИГИТАЛНИХ ДИПЛОМА У ОКВИРУ
ПЛАТФОРМЕ „КЕНГУР БЕЗ ГРАНИЦА“**

Ментор
др Марина Свичевић

Студент
Мирко Михајловић, 62/21

Јун 2026.

Садржај

Предговор	3
1. Увод.....	4
1.1. Мотивација и циљеви	4
1.2. Опис система пре имплементације	5
2. Преглед технологија	6
2.1. Клијентска страна	6
2.2. Серверска страна.....	8
3. Архитектура система	9
3.1. Преглед нових компоненти.....	9
3.2. MVC архитектура на серверској страни	10
3.3. Трофазни процес генерисања диплома.....	11
3.4. Шема базе података	12
3.4.1. Релације међу табелама	14
3.4.2. SQL скрипте за миграцију.....	16
3.5. Backend REST API	17
3.6. Серверски модели	19
3.7. Frontend сцене и компоненте	21
3.8. Клијентски сервис.....	27
3.9. Web Worker за рендеровање	28
4. Систем верификације диплома.....	32
5. Интернационализација и додатне измене.....	35
6. Обрада грешака и стабилност система	36
7. Безбедност система.....	37
8. Перформансе и оптимизације	38
9. Тестирање	39
10. Одржавање и унапређење система	40
11. Закључак	41
Литература.....	42
Кратка биографија кандидата	43

Предговор

Овај документ представља детаљну техничку документацију новог модула за управљање дипломама имплементираног у оквиру платформе „Кенгур без граница“. У склопу овог рада реализован је потпун систем за генерисање, верификацију и дистрибуцију диплома, чиме је дигитализован процес који је раније вршен искључиво ручно и ван платформе.

Систем обухвата генерисање диплома у реалном времену применом Web Workers технологије и OffscreenCanvas API-ја, систем верификације аутентичности диплома преко јединственог сертификационог кода, управљање шаблонима диплома путем CRUD интерфејса, административни преглед диплома по школама, те посебне странице намењене школским налозима за самостално преузимање диплома.

Поред главног модула за управљање дипломама, у оквиру истог пројектног циклуса извршен је и низ додатних побољшања постојеће платформе: унапређење административног сајдбара новим ставкама менија, проширење React рутирања новим заштићеним рутама, те интеграција нових компоненти у постојећи `il8next` систем интернационализације.

Систем је развијен применом React библиотеке на клијентској страни, у комбинацији са PHP и MySQL на серверској страни, пратећи MVC архитектурни образац. Нови модул је потпуно адитивне природе – нису измењени постојећи API endpoints, чиме је осигурана компатибилност са претходном верзијом платформе и омогућено постепено увођење нових функционалности без ризика по постојеће процесе.

1. Увод

Платформа „Кенгур без граница“ представља свеобухватни информациони систем за управљање највећим светским математичким такмичењем у коме учествује преко 120 земаља. Систем поприма улогу централног чворишта за организацију такмичења, евиденцију ученика, управљање школама, те обраду и приказ резултата такмичара свих узраста, од првог разреда основне до четвртог разреда средње школе. Због великог броја учесника и сложености организације, свака функционалност која аутоматизује ручне процесе има директан утицај на ефикасност читавог такмичења.

У оквиру даљег развоја платформе идентификована је критична потреба за дигитализацијом процеса издавања и верификације диплома. Традиционалан приступ ручног штампања и дистрибуције диплома поштом носи бројне изазове: временску неефикасност, могућност грешке при изради, тешкоћу верификације аутентичности, те значајне финансијске трошкове штампе и поштанских услуга. У случају такмичења са хиљадама учесника широм Србије, ручни процес постаје практично неодржив, а свака грешка у имену, школи или категорији захтева поновно штампање и слање документа.

Нови модул за управљање дипломама решава све наведене проблеме кроз потпуну дигиталну аутоматизацију. Администратори могу увести листу ученика путем Excel датотеке, систем аутоматски генерише персонализоване дипломе применом предефинисаних шаблона, сваки документ добија јединствени сертификациони код, а школе могу преузети дипломе у PDF формату директно са платформе.

1.1. Мотивација и циљеви

Примарна мотивација за развој овог модула произашла је из потребе такмичарске заједнице да у кратком временском периоду генерише велики број персонализованих диплома за хиљаде ученика широм земље. Ручни процес је неодржив при таквом обиму, а ризик од грешке при уносу података растао је пропорционално са бројем учесника. Поред тога, изостанак механизма верификације значио је да примаоци дипломе нису имали поуздан начин да потврде њену аутентичност.

Кључни циљеви дизајна новог система укључивали су:

- аутоматизацију генерисања диплома применом шаблона у реалном времену;
- криптографски јединствену верификацију аутентичности сваке дипломе;
- подршку за вишеструке типове диплома – такмичарске, захвалнице, финалне и школске;
- ефикасну клијентску обраду великих скупова података применом Web Workers технологије;
- интуитиван интерфејс за управљање шаблонима са приказом изгледа шаблона;
- систем за преузимање ZIP архива са дипломама по школи;
- јавну страницу верификације аутентичности доступну без пријаве.

1.2. Опис система пре имплементације

Пре имплементације новог модула платформа је садржала функционалности за регистрацију учесника такмичења, унос и преглед резултата, управљање школама и налозима корисника, подношење жалби, те приказ ранг листа. Систем је такође садржао модул за финално такмичење са повезивањем и прегледом радова. Дакле, постојала је комплетна инфраструктура за вођење такмичења, али је њен крајњи производ – диплома – остајао ван дигиталног тока.

Међутим, систем није имао никакав механизам за генерисање или управљање дипломама – овај процес одвијао се потпуно ван платформе, ручно и без могућности дигиталне верификације. База података садржала је табеле за ученике, школе, категорије и резултате, али нису постојале табеле за дипломе, шаблоне диплома, сертификационе кодове нити архивирање диплома. Такође, табела са подацима о ученицима није садржала колону за матичну школу ученика, потребну за оне ученике који се такмиче у школи коју не похађају. Ова ограничења директно су одредила обим измена базе података описаних у каснијим поглављима.

2. Преглед технологија

У наставку следи преглед свих технологија, библиотека и алата примењених у имплементацији система за управљање дипломама. Избор технологија усклађен је са постојећом архитектуром платформе „Кенгур без граница“, уз додавање нових библиотека тамо где је то захтевала специфичност нових функционалности.

2.1. Клијентска страна

React је библиотека отвореног кода за изградњу интерактивних корисничких интерфејса и представља основу читаве клијентске стране платформе. Цео нови модул имплементиран је искључиво применом функционалних компоненти у комбинацији са React Hooks–има, пратећи модерне препоруке за развој React апликација. Коришћени hooks укључују useState за управљање локалним стањем, useEffect за повезивање са API–јем, useCallback за мемоизацију функција, useContext за приступ глобалном стању и useRef за директне DOM операције. Свака сцена у модулу организована је као засебна, независна компонента са јасно дефинисаним интерфејсом props–ова, чиме се омогућава лако поновно коришћење и тестирање уз потпуно раздвајање презентационе од пословне логике. Овакав приступ олакшава и каснија проширења, јер се нове функционалности додају као нове компоненте без задирања у постојеће.

Material UI (@mui/material, верзија 5.x) коришћен је за визуелну компоненту интерфејса и обезбеђује доследан изглед усклађен са остатком платформе. За приказ листе диплома употребљена је MUI DataGrid компонента, која обезбеђује серверску пагинацију, сортирање по колонама и inline уређивање. Поред DataGrid–а, систем интензивно користи компоненте попут Dialog (за модалне форме), Autocomplete (за избор шаблона и школа), LinearProgress (за приказ напретка генерисања), Snackbar са Alert (за нотификације), Chip (за визуелно разликовање типова диплома), Paper, Button, IconButton и TextField.

За прилагођавање изгледа постојећим дизајн стандардима платформе примењено је стилизовање путем styled() API–ја, чиме се избегава директно писање inline CSS–а и омогућава централизовано управљање темом.

Библиотека jsPDF (верзија ^2.5.x, Табела 1) коришћена је искључиво за финалну фазу генерисања диплома – конвертовање готове JPEG слике у PDF документ. Намерно је одабран приступ у коме се цео визуелни садржај дипломе (текст, фонт, позиционирање, стилизација) прво рендерује на OffscreenCanvas платну у Web Worker–у, а тек потом се готова слика уметне у PDF применом jsPDF–а. Овај приступ је изабран јер је знатно лакше и прецизније управљати фонтовима, димензијама, позиционирањем текста и визуелним ефектима директно на платну него применом jsPDF–ових интерних метода за цртање текста, које имају ограниченију подршку за ћириличне фонтове и сложеније распореде.

При конверзији у JPEG примењена је компресија квалитета 0.7 (70% од оригиналног квалитета) применом параметра quality: 0.7 у позиву canvas.convertToBlob(). Ово

представља намерни компромис: квалитет остаје визуелно прихватљив за штампани документ, а величина датотеке и потрошња меморије значајно се смањују, што је критично при генерисању стотина диплома у једној серији. Систем аутоматски одређује оријентацију странице (портрет или пејзаж) поређењем односа страница слике са пропорцијом A4 формата ($1:\sqrt{2}$), а слика се прецизно позиционира применом letterbox алгоритма тако да попуни целу A4 страницу без деформације или губитка садржаја.

Табела 1: Библиотека jsPDF и њена улога у генерисању диплома.

Библиотека	Верзија	Употреба	Важна напомена
jsPDF	^2.5.x	Конверзија OffscreenCanvas слика у PDF Blob	Аутоматски детектује оријентацију на основу односа страница слике

Web Workers и OffscreenCanvas API представљају критичан технолошки избор у архитектури система. Генерисање велике количине слика (потенцијално стотине или хиљаде диплома у једној серији) захтева значајне рачунарске ресурсе; ако би се ова обрада одвијала у главној UI нити, апликација би потпуно замрзла за све то време, онемогућавајући кориснику праћење напретка или вршење других операција.

Web Worker (diplomaWorker.js) извршава се у засебној нити оперативног система, прима листу ученика и bitmap шаблонске слике, генерише персонализовану дипломску слику за сваког ученика применом OffscreenCanvas API-ја и враћа резултате у виду ArrayBuffer-а. OffscreenCanvas је посебна имплементација Canvas-а доступна ван главне нити, што омогућава да се цело рендеровање одвија у позадини. Трансфер ArrayBuffer-а врши се применом Transferable Objects механизма, чиме се избегава скупо копирање меморије између нити.

SheetJS (xlsx) коришћен је за парсирање Excel датотека (.xlsx формат) које администратори читавају са листом ученика. Систем чита наслове колона из Excel датотеке и мапира их на интерне структуре података применом флексибилног мапирања које подржава и ћириличне и латиничне варијанте назива колона. Систем такође проверава да ли учитана датотека садржи резултате за очекивану такмичарску годину и приказује упозорење ако се годишта не подударају, чиме се спречава случајно генерисање диплома на основу погрешне датотеке.

Библиотека FileSaver.js користи се за преузимање датотека у веб-прегледачу и обезбеђује исправан рад у различитим прегледачима применом Blob API механизма. Коришћен је за иницирање преузимања ZIP архива са дипломама, PDF докумената и Excel извештаја директно из клијентске апликације, без потребе за преусмеравањем корисника на URL адресу сервера. Библиотека решава ограничења различитих прегледача у погледу покретања преузимања и обезбеђује јединствено понашање апликације на различитим платформама и оперативним системима.

2.2. Серверска страна

PHP – REST API. Серверска страна новог модула имплементирана је у програмском језику PHP, пратећи исту MVC архитектурну шему коришћену у преосталом делу платформе. За сваки нови endpoint постоји одговарајући контролер, модел и рута. PHP контролери обрађују HTTP захтеве, валидирају улазне податке провером типова и вредности, позивају методе модела преко DB.php класе и враћају одговоре клијенту у JSON формату. Доследна примена истог обрасца у целом систему олакшава одржавање, јер сваки нови endpoint прати исту, унапред познату структуру.

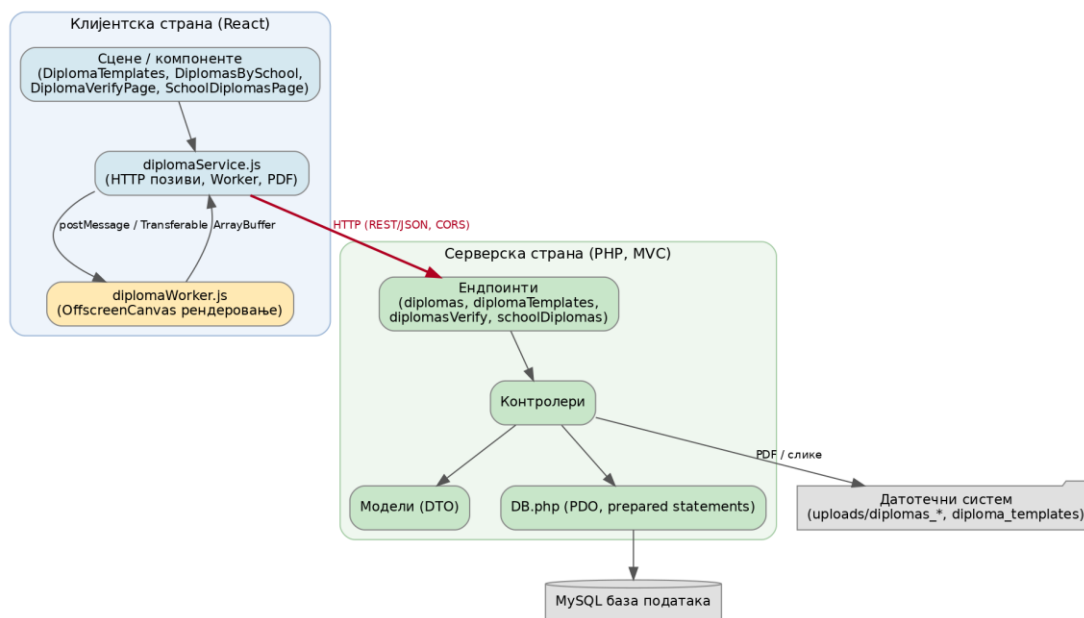
MySQL – база података. Серверска страна ослања се на MySQL базу података за трајно чување свих метаподатака о дипломама, шаблонима и сертификационим кодовима. За све интеракције са базом коришћена је постојећа DB.php класа са PDO (PHP Data Objects) механизмом и припремљеним упитима (prepared statements) за заштиту од SQL injection напада. Параметри се прослеђују у облику асоцијативних низова са именованим параметрима (:param_name), што истовремено повећава читљивост упита и спречава мешање корисничких података са структуром SQL наредбе.

PHP ZipArchive. Уграђена PHP класа ZipArchive коришћена је за генерисање ZIP архива са PDF дипломама директно на серверској страни. Функционалности за масовно преузимање диплома по школи, по типу дипломе и за цео систем ослањају се на ову класу. Дипломе се пакују са описним именима датотека која укључују тип дипломе, назив ученика и такмичарску годину, чиме се олакшава сналажење у преузетим архивама. Серверско генерисање ZIP архиве знатно је ефикасније у односу на клијентско јер избегава трансфер великих количина бинарних података преко мреже.

3. Архитектура система

Нови модул за управљање дипломама пројектован је применом слојевите архитектуре која јасно раздваја одговорности различитих компоненти система. На клијентској страни примењен је принцип раздвајања на сцене (pages), сервисе и помоћне модуле, док се на серверској страни следи MVC образац са контролерима, моделима и endpoint рутама. Ова архитектура осигурава лако одржавање, тестирање и проширивање система. Општи приказ слојева и њихове међусобне комуникације дат је на Слика 1.

Слојевитост архитектуре није сама себи циљ, већ практично средство за управљање сложености. Раздвајањем система на јасно одвојене слојеве – приказ, пословна логика и приступ подацима – постиже се да се измена у једном слоју може спровести са минималним утицајем на остале.



Слика 1: Слојевита архитектура система са приказом комуникације између клијентске и серверске стране.

3.1. Преглед нових компоненти

Систем за управљање дипломама уводи потпуно нови скуп компоненти на обе стране апликације. Списак нових датотека на клијентској страни приказан је у Табели 2, а на серверској страни у Табели 3. Из приказа се види да је модул, иако функционално обиман, организован у мали број јасно одвојених датотека са прецизно дефинисаним одговорностима.

Табела 2: Нове датотеке на клијентској (frontend) страни.

Тип	Нова датотека
API сервис	src/api/diplomaService.js
Web Worker	src/workers/diplomaWorker.js

Тип	Нова датотека
Сцена: Шаблони	src/scenes/diplomaTemplates/index.js
Дијалог: Додај шаблон	src/scenes/diplomaTemplates/addDiplomaTemplateDialog.js
Сцена: Дипломе по школи (admin)	src/scenes/diplomasBySchool/index.js
Дијалог: Увоз диплома	src/scenes/diplomasBySchool/importDiplomaDialog.js
Дијалог: Увоз захвалница	src/scenes/diplomasBySchool/importZahvalnicaDialog.js
Сцена: Верификација	src/scenes/diplomaVerifyPage/index.js
Сцена: Дипломе (школски налог)	src/scenes/schoolDiplomasPage/index.js

Табела 3: Нове датотеке на серверској (backend) страни.

Тип	Нова датотека
Endpoint	api/diplomas.php
Endpoint	api/diplomaTemplates.php
Endpoint	api/diplomasVerify.php
Endpoint	api/schoolDiplomas.php
Контролер	api/controllers/diplomasController.php
Контролер	api/controllers/diplomaTemplatesController.php
Контролер	api/controllers/diplomaVerifyController.php
Контролер	api/controllers/schoolDiplomasController.php
Модел	api/models/diploma.php
Модел	api/models/diplomaTemplate.php
Модел	api/models/diplomaVerify.php
Модел	api/models/studentDiploma.php

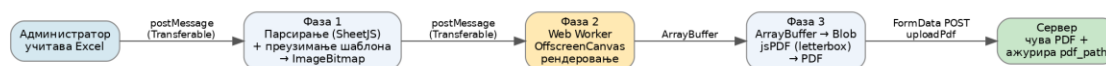
3.2. MVC архитектура на серверској страни

Серверска страна новог модула прати исти MVC образац примењен у преосталом делу система. Свака рута (endpoint PHP датотека) инстанцира одговарајући контролер, а контролер позива методе преко DB.php класе. Модели служе искључиво као структуре података (Data Transfer Objects) без имплементиране пословне логике, што значи да се сва логика обраде налази у контролерима, а структура података у моделима.

Ток обраде сваког HTTP захтева пролази кроз неколико слојева. Клијент најпре шаље HTTP захтев (GET, POST, PUT или DELETE) на одговарајући PHP endpoint. Endpoint датотека укључује CORS механизам и инстанцира контролер, који затим валидира параметре, позива методе DB.php класе и конструише одговор. Контролер враћа json_encode() одговор клијенту, након чега клијентски код ажурира стање компоненте (useState) и приказује нове податке.

3.3. Трофазни процес генерисања диплома

Генерисање диплома на клијентској страни одвија се кроз трофазни процес: припрему података, рендеровање на платну у позадинској нити и конверзију у PDF формат са отпремањем на сервер. Свака фаза јасно је одвојена од осталих, што олакшава праћење тока и локализацију евентуалних грешака. Поједностављен ток кроз ове три фазе приказан је на Слика 2.



Слика 2: Поједностављен ток трофазног процеса генерисања дипломе, од учитавања Excel датотеке до чувања PDF документа на серверу.

У **првој фази** администратор учитава Excel датотеку са листом ученика применом стандардног `file input` елемента. Систем парсира датотеку применом SheetJS библиотеке и конструише листу записа ученика у стандардизованом формату – SheetJS чита прву страну Excel радне свеске, издваја заглавља из прве врсте и мапира сваки ред на интерни DTO објекат.

Паралелно са парсирањем, систем преузима слику шаблонске дипломе са сервера путем функције `diplomaService.getTemplateImageUrl()` и конвертује је у `ImageBitmap` применом `createImageBitmap()`; овај корак је неопходан јер Web Worker не може директно да приступа DOM-у нити да учитава URL адресе преко HTTP-а.

У **другој фази** листа ученика и `bitmap` шаблона прослеђују се Web Worker-у путем позива `postMessage()` са `Transferable Objects`. Пренос `ImageBitmap`-а представља трансфер без копирања – власништво над меморијским блоком преноси се из главне нити у Worker, чиме се избегава дупликација меморије.

Worker затим, за сваког ученика у листи, наноси текст на шаблонску слику следећим редоследом: прво се цела шаблонска слика исцрта на платно применом `drawImage()`; потом се постављају параметри фонта (Cambria или Times New Roman); функција `drawWrappedText()` наноси сваки текстуални елемент са аутоматским преламањем линија и белим контурним ефектом (`strokeText` пре `fillText`) ради читљивости на шареним позадинама; на крају се платно конвертује у `ArrayBuffer` методом `convertToBlob()` и враћа главној нити.

Систем при рендеровању разликује два типа диплома: стандардне дипломе (тип 2 и 3) садрже пуно име ученика, разред, назив школе, матичну школу, место и назив награде, док школске дипломе (тип 1 и 4 – Златна захвалница и Захвалница) садрже само назив школе и место. Оба типа садрже сертификациони код и верификациони URL.

У **трећој фази** главна нит прима резултате за сваког ученика и конвертује `ArrayBuffer` у `Blob` слике применом `new Blob()`. Применом `jsPDF` библиотеке

генерише се PDF документ: систем прво одређује оријентацију A4 странице поређењем односа страница слике са пропорцијом $1:\sqrt{2}$, затим израчунава позицију и димензије слике унутар странице применом letterbox алгоритма (слика се скалира да попуни максималан простор уз очување размере), па се слика додаје у PDF применом doc.drawImage().

Готов PDF Blob шаље се на сервер путем FormData POST захтева на api/diplomas.php?action=uploadPdf. Сервер чува датотеку у структурирани директоријум (uploads/diplomas_{година}/) и ажурира поље pdf_path у бази података. У случају регенерације дипломе, стари PDF се физички брише пре уписа новог.

3.4. Шема базе података

Имплементација система за управљање дипломама захтевала је додавање четири нове табеле у постојећу MySQL базу података. Нове табеле дизајниране су са циљем минималног утицаја на постојећу структуру. Поред додавања потпуно нових табела, постојећа табела student проширена је новом колоном maticna_skola (VARCHAR(255)), неопходном за приказ матичне школе ученика на дипломама.

Табела diploma_templates (Табела 4) чува дефиниције свих шаблона диплома. Сваки шаблон дефинише тип дипломе, такмичарску годину и путању до слике шаблона на серверу, при чему слика представља позадину на коју систем приликом генерисања наноси персонализовани текст. На овај начин се исти визуелни предлогак може поново користити за велики број диплома исте врсте.

Табела 4: Структура табеле diploma_templates.

Колона	Тип	NULL	Опис
id	INT(11) AUTO_INCREMENT	NOT NULL	Примарни кључ
name	VARCHAR(255)	NOT NULL	Назив шаблона
type	INT(11)	NOT NULL	Тип: 1=Златна, 2=I ранг, 3=Финале, 4=Захвалница
year	INT(11)	NOT NULL	Такмичарска година
image	VARCHAR(255)	NULL	Путања до слике шаблона

Централна табела diplomas (Табела 5) чува метаподатке о свакој генерисаној дипломи. Свака врста представља издату диплому по школи или ученику, са референцом на шаблон и путањом до генерисане PDF датотеке. Табела подржава и школске дипломе (Златна захвалница, Захвалница) кроз nullable колону student_id и поља excel_school_name и excel_school_location која се користе за Златну захвалницу. Тиме је иста табела довољно општа да покрије и дипломе везане за појединачног ученика и дипломе које се односе на школу као институцију.

Табела 5: Структура централне табеле *diplomas*.

Колона	Тип	NULL	Опис
id	INT(11) AUTO_INCREMENT	NOT NULL	Примарни кључ
student_id	INT(11)	NULL	FK → student.id (NULL за школске дипломе)
school_id	INT(11)	NOT NULL	FK → school.id
award	VARCHAR(50)	NULL	Тип награде: I, II, III, Похвала, Захвалница, Златна
excel_school_name	VARCHAR(500)	NULL	Назив школе из Excel датотеке
excel_school_location	VARCHAR(500)	NULL	Место школе из Excel датотеке
template_id	INT(11)	NULL	FK → diploma_templates.id
pdf_path	TEXT	NULL	Путања до PDF документа на серверу
created_at	TIMESTAMP	NOT NULL	Датум и време креирања
updated_at	TIMESTAMP	NOT NULL	Датум и време ажурирања
year	SMALLINT(5) UNSIGNED	NULL	Такмичарска година

Табела *diploma_certificates* (Табела 6) садржи јединствене сертификационе кодове за сваку диплому и омогућава верификацију аутентичности. Свака диплома добија јединствени алфанумерички код формата XXXXXXXX-*{diploma_id}* приликом генерисања, а однос са табелом *diplomas* строго је 1:1 – свака диплома може имати тачно један сертификациони код. Овакав однос гарантује да се на основу кода увек једнозначно идентификује тачно једна диплома.

Табела 6: Структура табеле *diploma_certificates*.

Колона	Тип	NULL	Опис
id	INT AUTO_INCREMENT	NOT NULL	Примарни кључ
diploma_id	INT	NOT NULL	FK → diplomas.id (UNIQUE, строго 1:1)
certification_id	VARCHAR(50)	NOT NULL	Јединствени код (нпр. AB3X7Q2N-42), UNIQUE
student_id	INT	NULL	FK → student.id (NULL за школске дипломе)
school_id	INT	NOT NULL	FK → school.id

Колона	Тип	NULL	Опис
created_at	TIMESTAMP	NOT NULL	Датум и време издавања сертификата

Табела `archived_school_diplomas` (Табела 7) чува архивирани записе о дипломама уклоњеним из главне табеле `diplomas`. Она омогућава историјски увид у претходно издате дипломе без трајног губитка података. Архивирање се врши приликом брисања или замене дипломе, чиме се обезбеђује потпун ревизорски траг (енг. *audit trail*) и могућност накнадног праћења свих измена над подацима. Захваљујући овој табели, ниједан податак о некада издатој дипломи не нестаје из система.

Табела 7: Структура табеле `archived_school_diplomas`.

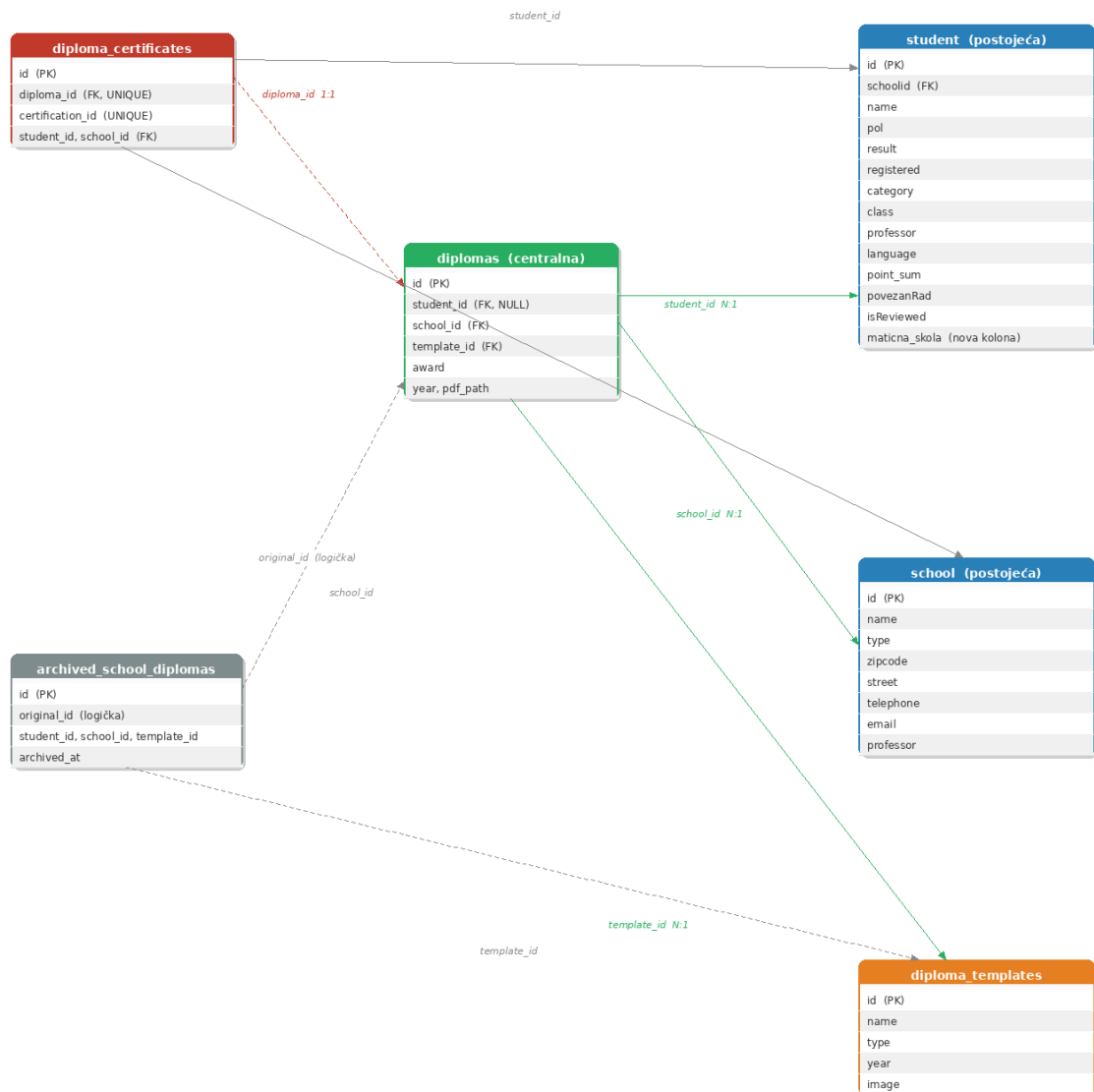
Колона	Тип	NULL	Опис
id	INT(10) UNSIGNED	NOT NULL	Примарни кључ архивираног записа
original_id	INT(10) UNSIGNED	NOT NULL	ID оригиналног записа из табеле <code>diplomas</code>
student_id	INT(10) UNSIGNED	NULL	FK → <code>student.id</code> (NULL за школске дипломе)
school_id	INT(10) UNSIGNED	NULL	FK → <code>school.id</code>
award	VARCHAR(50)	NULL	Тип награде
template_id	INT(10) UNSIGNED	NULL	FK → <code>diploma_templates.id</code>
pdf_path	VARCHAR(500)	NULL	Путања до PDF датотеке у тренутку архивирања
excel_school_name	VARCHAR(255)	NULL	Назив школе из Excel датотеке
excel_school_location	VARCHAR(255)	NULL	Место школе из Excel датотеке
year	SMALLINT(5) UNSIGNED	NULL	Такмичарска година
created_at	DATETIME	NULL	Оригинални датум креирања дипломе
updated_at	DATETIME	NULL	Оригинални датум последњег ажурирања
archived_at	DATETIME	NOT NULL	Датум архивирања

3.4.1. Релације међу табелама

Нове табеле повезане су са постојећим табелама базе кроз стране кључеве. Табела `diplomas` је централна и повезана је са табелама `student` (`student_id`), `school` (`school_id`) и `diploma_templates` (`template_id`). Табела `diploma_certificates` је у строгом односу 1:1 са табелом `diplomas`, што значи да свака диплома може имати тачно један

сертификациони код. Целовит приказ релација између нових и постојећих табела дат је на Слика 3.

Од постојећих табела измењена је искључиво табела student, којој је додата колона maticna_skola (VARCHAR(255)) за потребе приказа матичне школе на дипломама. Све остале постојеће табеле остале су потпуно непромењене, чиме је обезбеђена потпуна компатибилност са претходном верзијом платформе и омогућено да стара функционалност настави да ради непромењено чак и током постепеног увођења новог модула.



Слика 3: Шема релација између нових и постојећих табела базе података

3.4.2. SQL скрипте за миграцију

За активирање новог модула потребно је извршити SQL скрипте које креирају нове табеле, као и скрипту која проширује постојећу табелу student.

```
CREATE TABLE diploma_templates (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(255) NOT NULL,  
  type    TINYINT NOT NULL,  
  year    YEAR NOT NULL,  
  image   VARCHAR(512)  
);  
  
CREATE TABLE diplomas (  
  id              INT AUTO_INCREMENT PRIMARY KEY,  
  student_id      INT,  
  school_id       INT NOT NULL,  
  template_id     INT,  
  award           VARCHAR(50),  
  year            YEAR,  
  pdf_path        VARCHAR(512),  
  excel_school_name VARCHAR(255),  
  excel_school_location VARCHAR(255),  
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);  
  
CREATE TABLE diploma_certificates (  
  id              INT AUTO_INCREMENT PRIMARY KEY,  
  diploma_id      INT NOT NULL UNIQUE,  
  certification_id VARCHAR(50) NOT NULL UNIQUE,  
  student_id      INT,  
  school_id       INT NOT NULL,  
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE archived_school_diplomas (  
  id              INT AUTO_INCREMENT PRIMARY KEY,  
  original_id     INT,  
  student_id      INT,  
  school_id       INT NOT NULL,  
  template_id     INT,  
  award           VARCHAR(50),  
  year            YEAR,  
  pdf_path        VARCHAR(512),  
  excel_school_name VARCHAR(255),  
  excel_school_location VARCHAR(255),  
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  archived_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
ALTER TABLE student  
  ADD COLUMN maticna_skola VARCHAR(255) NULL;
```

3.5. Backend REST API

Серверска страна система за управљање дипломама излаже четири REST API endpoint-a. Сваки endpoint имплементиран је у складу са REST конвенцијама, враћа одговоре у JSON формату и укључује CORS заглавља за прихватање захтева са клијентске React апликације.

Endpoint за шаблоне (api/diplomaTemplates.php) обрађује CRUD операције над шаблонима диплома и подржава отпремање слика шаблона у multipart/form-data формату. Дозвољене методе и акције приказане су у Табели 8.

Табела 8: Методе и акције endpointa api/diplomaTemplates.php.

HTTP метода	Акција / Параметри	Опис
GET	(без параметара)	Враћа листу свих шаблона сортирану по години (DESC)
GET	?action=get_image&id={id}	Директно служи бинарну слику шаблона (Content-Type: image/png)
POST	multipart: name, type, year, image (File)	Креира нови шаблон дипломе са учитаном сликом
POST	?id={id}&_method=PUT (multipart)	Ажурира постојећи шаблон (замена слике опциона)
DELETE	?id={id}	Брише шаблон и физичку датотеку слике са диска

Структура JSON одговора за успешан GET захтев (листа шаблона) има следећи облик:

```
[
  {
    "id": 1,
    "name": "Diploma prvog ranga 2025",
    "type": 2,
    "year": 2025,
    "image": "uploads/diploma_templates/1712345678_template.png"
  }
]
```

Централни endpoint за дипломе (api/diplomas.php) подржава масовно генерисање (batch create из Excel увоза), индивидуалну регенерацију постојећих диплома, преузимање ZIP архива, провере стања и статистику. Дозвољене методе и акције приказане су у Табели 9. Будући да обједињује већину операција над дипломама, овај endpoint је најобимнији у систему.

Табела 9: Методе и акције endpointa *api/diplomas.php*.

HTTP метода	Акција / Параметри	Опис
GET	(без параметара)	Враћа листу свих диплома у систему
GET	?action=getBySchool&school_id={id}	Враћа дипломе за школу са детаљима ученика
GET	?action=downloadZip&school_id={id}	Генерише и испоручује ZIP архиву школе
GET	?action=downloadAllDiplomas	ZIP архива са свим дипломама у систему (admin)
GET	?action=downloadAllResults	ZIP архива са свим ZIP-овима по школама
GET	?action=regenerateSingle&studentId={id}	Враћа метаподатке за регенерацију дипломе
GET	?action=checkResults	Проверава да ли постоје резултати у систему
GET	?action=countsBySchool	Броји дипломе по школи (за статистику)
GET	?action=diplomaYears	Листа доступних такмичарских година
GET	?action=zahvalnicaSchools	Листа школа које немају захвалницу
POST	?action=bulkCreate	Серијско креирање диплома (из Excel увоза)
POST	?action=uploadPdf	Отпремање генерисаног PDF документа дипломе
POST	?action=uploadPdf®enerate=1	Отпремање регенерисаног PDF и поништавање старог
POST	?action=saveResult	Чување резултатске датотеке

Endpoint за јавну верификацију (*api/diplomasVerify.php*) не захтева аутентификацију – доступан је свим корисницима без пријаве и прихвата искључиво GET захтеве, како је приказано у Табели 10.

Табела 10: Метода и акција endpointa *api/diplomasVerify.php*.

HTTP метода	Акција / Параметри	Опис
GET	?id={certificationId}	Верификује сертификациони код и враћа потпуне детаље дипломе

Пример успешног одговора за стандардну диплому ученика и одговора за неважећи сертификациони код:

```
{
  "valid": true,
  "certification_id": "AB3X7Q2N-42",
  "student_name": "Марко Петровић",
}
```

```

"class": 5,
"school_name": "ОШ Душан Радовић",
"diploma_year": 2025,
"template_type": "2",
"type_label": "Diploma prvog ranga",
"award": "I",
"is_school_level": false
}

{ "valid": false, "error": "Diploma not found" }

```

Endpoint за школске налоге (api/schoolDiplomas.php) намењен је школским налозима за преглед и преузимање сопствених диплома и прихвата искључиво GET захтеве. Свим SQL упитима додат је филтер WHERE d.school_id = :school_id, чиме се осигурава да школа може приступити искључиво сопственим дипломама. Акције су приказане у Табели 11.

Табела 11: Методе и акције endpointa api/schoolDiplomas.php.

HTTP метода	Акција / Параметри	Опис
GET	?action=list&school_id={id}	Листа диплома школе (опционо &class={c}&year={y})
GET	?action=classes&school_id={id}	Доступни разреди за школу
GET	?action=downloadSingle&school_id={id}&student_id={id}	Преузимање PDF дипломе ученика
GET	?action=downloadZlatna&school_id={id}&year={y}	Преузимање PDF Златне захвалнице школе
GET	?action=downloadZahvalnica&school_id={id}&year={y}	Преузимање PDF Захвалнице школе
GET	?action=downloadAll&school_id={id}&year={y}	ZIP архива са свим дипломама школе за годину

3.6. Серверски модели

Серверски слој модела дефинише PHP класе које представљају структуре података у апликацији. Модели не садрже пословну логику – служе искључиво као Data Transfer Objects (DTO) за безбедан пренос података између слојева апликације. Овакав приступ осигурава да свака промена у структури базе захтева измену само у одговарајућем моделу, без потребе за изменом контролера или endpointa, што додатно учвршћује раздвајање одговорности у систему.

Класа *Diploma* представља основну структуру генерисане дипломе; њене инстанце конструишу се у методи *rowToDiploma()* контролера *DiplomasController* на основу редова из MySQL упита. Атрибути ове класе приказани су у Табели 12.

Табела 12: Атрибути модела Diploma.

Атрибут	Опис
\$id	Примарни кључ из табеле <i>diplomas</i>
\$student_id	ID ученика (NULL за школске дипломе – Златна/Захвалница)
\$school_id	ID школе (увек присутан)
\$class	Разред ученика (1–12); добија се JOIN–ом са табелом <i>student</i> , не чува се директно у <i>diplomas</i>
\$award	Тип награде: I, II, III, Похвала, Златна, Захвалница
\$template_id	ID шаблона дипломе из табеле <i>diploma_templates</i>
\$pdf_path	Релативна путања до PDF документа (NULL пре генерисања)
\$excel_school_name	Назив школе из Excel датотеке (за захвалнице без школе у бази)
\$excel_school_location	Место школе из Excel датотеке (за захвалнице)
\$year	Такмичарска година на коју се диплома односи
\$created_at	Временска ознака креирања (аутоматски)
\$updated_at	Временска ознака последњег ажурирања (аутоматски)

Класа *DiplomaTemplate* представља шаблон дипломе. Систем дефинише четири дозвољене вредности за поље *type*, приказане у Табели 13.

Табела 13: Дозвољене вредности поља type у моделу DiplomaTemplate.

Вредност type	Назив шаблона	Намена
1	Златна захвалница	Школска диплома за школе са изузетним резултатима
2	Диплома I ранга	Такмичарска диплома за ученике са I наградом
3	Диплома финалног ранга	Такмичарска диплома за финалисте
4	Захвалница	Школска диплома за учешће школе

Класа *DiplomaVerify* представља запис у табели *diploma_certificates* и садржи атрибуте *id*, *diplomaId*, *certificationId*, *studentId*, *schoolId* и *createdAt*. Њене инстанце конструишу се у методи *rowToVerify()* контролера *DiplomaVerifyController*; оне се не користе директно за JSON серијализацију јер контролер из безбедносних разлога враћа сирове низове добијене JOIN упитом.

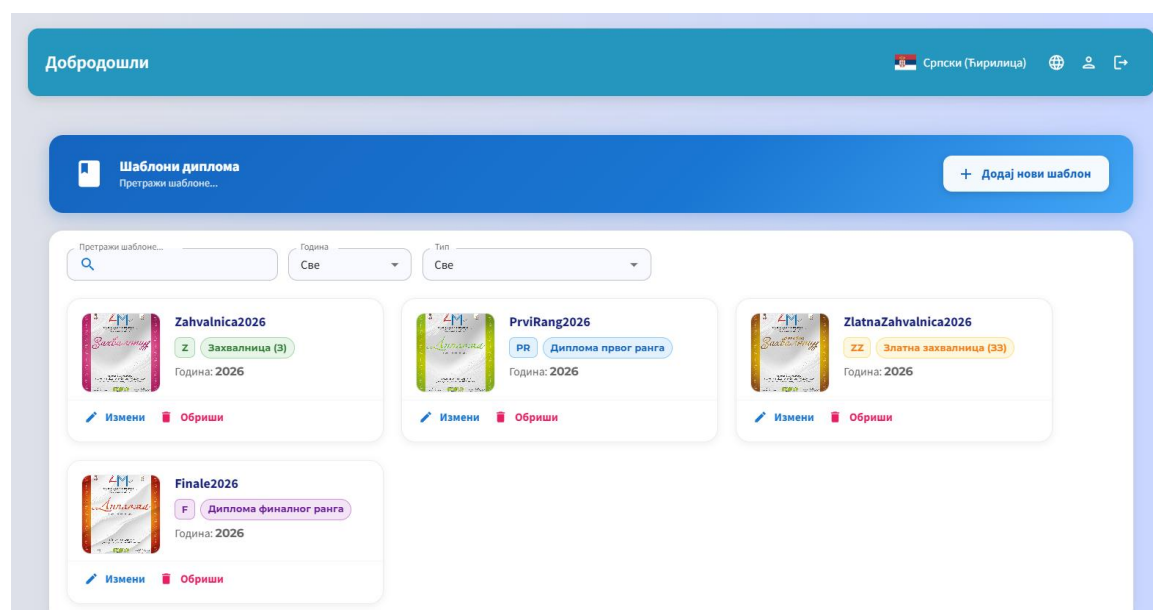
Класа *StudentDiploma*, насупрот томе, представља проширени приказ дипломе намењен школском прегледу у компоненти *SchoolDiplomasPage* – поред основних атрибута дипломе садржи и денормализоване атрибуте добијене JOIN операцијама

(student_name, school_name, location_name, category_name, professor, language и point_sum), уз имплементирану методу toArray() за laku JSON серијализацију целог објекта. Овакво раздвајање два модела дипломе омогућава да се исти основни ентитет приказује на различите начине у зависности од контекста и улоге корисника.

3.7. Frontend сцене и компоненте

Клијентски модул за управљање дипломама организован је у четири главне сцене (странице) и три дијалога за увоз. Свака сцена имплементирана је као независна React функционална компонента стилизована применом Material UI-a, а сцене су регистроване у React Router конфигурацији у датотеци App.js са одговарајућим заштићеним рутама.

Управљање шаблонима диплома. Страница DiplomaTemplates намењена је административним корисницима и омогућава централизовано управљање шаблонима диплома. Као што је приказано на Слика 4, кориснику је омогућен преглед свих доступних шаблона заједно са информацијама о типу дипломе и такмичарској години. Систем подржава претрагу и филтрирање шаблона према различитим критеријумима, чиме се олакшава проналажење одговарајућих шаблона у случају већег броја доступних записа.



Слика 4: Сцена за управљање шаблонима диплома (DiplomaTemplates) – картични приказ и претрага

Поред прегледа постојећих шаблона, страница омогућава додавање нових, измену постојећих и брисање шаблона који више нису потребни. Кориснику је такође омогућен увећани приказ шаблона ради лакшег прегледа њиховог изгледа и садржаја. Након извршавања операција систем приказује одговарајуће поруке о

успешности или евентуалним грешкама, чиме се кориснику пружа повратна информација о резултату извршене акције.

Систем шаблона диплома омогућава администраторима потпуну контролу над визуелним изгледом генерисаних диплома без потребе за модификацијом програмског кода. Нови шаблон може се учитати путем веб интерфејса у формату PNG или JPG слике, а систем аутоматски одређује резолуцију и оријентацију шаблона на основу димензија учитане слике. Сlike шаблона чувају се на серверу у посебном директоријуму, при чему систем приликом отпремања генерише јединствен назив датотеке како би се избегли конфликти приликом складиштења.

Приликом измене шаблона систем води рачуна о интегритету података тако што аутоматски уклања старе слике које више нису у употреби, уз претходну проверу да исту датотеку не користи више шаблона. На тај начин спречава се настанак неважећих референци и непотребно заузимање простора на серверу.

Шаблони се корисничком интерфејсу приказују у облику картица, при чему је сваки тип дипломе визуелно означен одговарајућом MUI Chip компонентом ради лакшег разликовања. Изабрана слика шаблона учитава се са сервера путем наменског API позива, што омогућава њено безбедно приказивање у прегледачу и коришћење приликом генерисања диплома.

Додавање и измена шаблона реализовани су путем посебног дијалога AddDiplomaTemplateDialog. У оквиру овог дијалога корисник уноси основне податке о шаблону, укључујући назив, тип дипломе и такмичарску годину, као и одговарајућу графичку подлогу која ће бити коришћена током генерисања диплома. Систем врши проверу унетих података пре чувања како би се обезбедила исправност и потпуност информација неопходних за даље коришћење шаблона.

Раздвајањем визуелног шаблона од података који се на њега наносе током генерисања дипломе, обезбеђена је висока флексибилност система. Организатор може мењати дизајн постојећих диплома или уводити нове типове једноставним додавањем новог шаблона кроз кориснички интерфејс, без потребе за изменама у имплементацији система.

Дијалози за увоз. Два модална дијалога служе за увоз диплома из Excel датотека: ImportDiplomaDialog за стандардне такмичарске дипломе (тип 2 и 3) и ImportZahvalnicaDialog за захвалнице на нивоу школе (тип 4). Оба дијалога имплементирају исти основни ток – учитавање Excel датотеке применом SheetJS парсирања, приказ прегледа учитаних података, избор шаблона уз филтрирање по типу и години, упозорење ако се година у датотеци не подудара са шаблоном, и покретање генерисања (Слика 5). Овакав предвидив редослед корака смањује могућност грешке администратора при уносу.

Изаберите шаблон дипломе

Обавезно: фајл мора бити у Excel формату (.xlsx или .xls)
 Морате изабрати шаблон дипломе и Excel фајл да бисте креирали дипломе. Excel фајл служи само за идентификацију ученика — подаци се преузимају из базе.

Дипломе за 2026. годину (Диплома првог ранга) већ постоје!
 Тренутно постоји 3117 генерисаних диплома овог типа за ову годину. Наставком увоза постојеће дипломе ће бити замењене новим.

Година: Све Тип: Све



PrviRang2026
 Година: 2026
 Тип: Диплома првог ранга



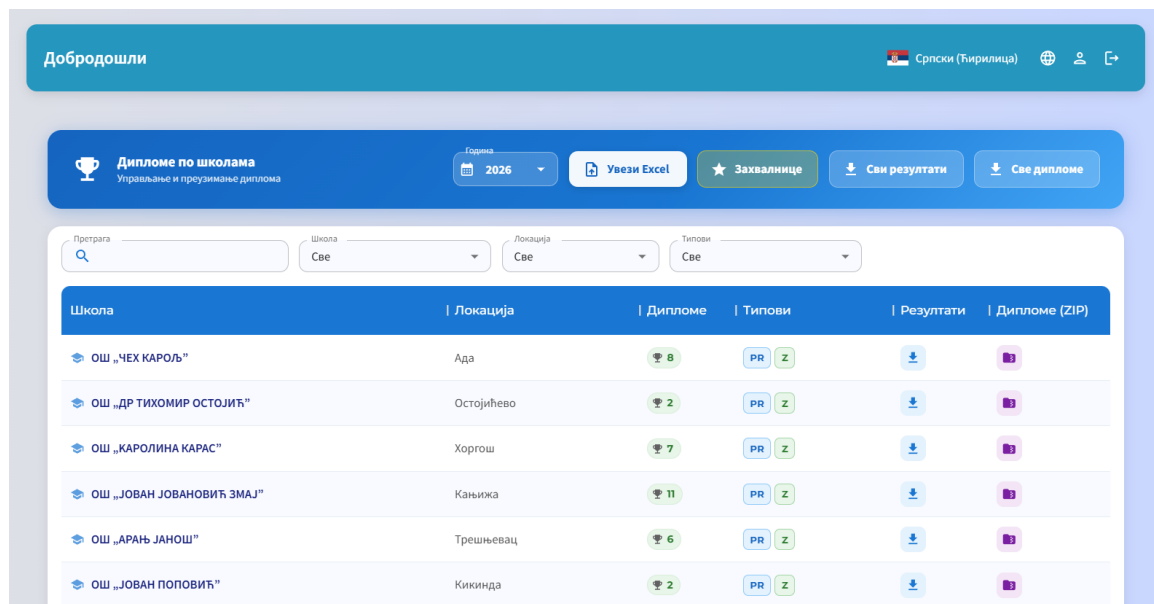
Finale2026
 Година: 2026
 Тип: Диплома финалног ранга

Изабери Excel фајл

ОТКАЖИ ЗАМЕНИ И УВЕЗИ

Слика 5: Дијалог за увоз диплома из Excel датотеке (ImportDiplomaDialog) – преглед учитаних података и избор шаблона.

Административни преглед и генерисање. Најсложенија сцена у систему, доступна искључиво административним налозима, представља централну управљачку таблу за генерисање и праћење диплома по школама (Слика 6). Она повезује процес увоза Excel датотеке, покретање генерисања у Web Worker–у, праћење напретка у реалном времену и складиштење резултата на серверу. Компонента управља значајним бројем React useState hook–ова организованих у логичке групе – подаци о дипломама (rows, templates, resultsExist, diplomasExist), филтрирање (searchText, filterSchool, filterLocation, filterType, selectedYear), стање увоза диплома и захвалница, статуси генерисања и нотификације.



Слика 6: Административна сцена за преглед и генерисање диплома по школама (DiplomasBySchool).

Генерисање диплома је асинхрони вишефазни процес покренут кликом на дугме „Генериши дипломе“. Систем најпре учитава слику шаблона са сервера применом `getTemplateImageUrl()` и конвертује је у `ImageBitmap`, будући да `Web Worker` не може директно да учитава слике преко `HTTP`-а. Пре генерисања `PDF` диплома, за сваку школу се креира и чува `.xlsx` датотека са резултатима ученика (разред, име, назив школе, место, категорија, професор, језик, поени и награда), формирана применом `SheetJS` библиотеке и послата на сервер путем `POST` захтева на `api/diplomas.php?action=saveResult`; ове датотеке касније су доступне за преузимање по школи или масовно као `ZIP` архива. Уместо једног `Worker`-а, систем покреће онолико инстанци колико процесор подржава паралелних нити (`navigator.hardwareConcurrency`, подразумевано 8), чиме се омогућава паралелно генерисање за различите школе и разреде. Листа ученика дели се у блокове од највише 100 ученика и прослеђује `Worker`-има путем `postMessage()` са `Transferable Objects` механизмом, при чему сваки `Worker` добија своју листу и исту шаблонску `bitmap` слику. `Worker` за сваког ученика исцртава шаблон на платно, наноси персонализован текст и конвертује резултат у `JPEG` са квалитетом 0.7, враћајући `ArrayBuffer` главној нити. Главна нит затим одређује оријентацију `A4` странице и генерише `PDF` применом `jsPDF`-а са `letterbox` позиционирањем, након чега се готов `PDF` отправља на сервер заједно са метаподацима. По завршетку процеса `Worker`-и се гасе применом `worker.terminate()` ради ослобађања ресурса. Поједностављена логика поделе посла на више `Worker` инстанци изгледа овако:

```
const poolSize = navigator.hardwareConcurrency || 8;
const chunks = splitIntoChunks(students, 100);
let next = 0;
const workers = Array.from({ length: poolSize }, () => createWorker());
await Promise.all(workers.map((w) => (async () => {
  while (next < chunks.length) {
    const chunk = chunks[next++];
```

```

    const buffers = await renderChunk(w, chunk, templateBitmap);
    await uploadResults(buffers);
  }
  w.terminate();
}));

```

За време трајања генерисања, дијалози за увоз приказују полупрозорни loading overlay са CircularProgress анимацијом и пратећом поруком. Дијалог се у том периоду не може затворити – позив onClose је блокиран (onClose={loading ? undefined : onClose}), а дугмад за отказивање и потврду су онемогућена, чиме се спречава случајан прекид генерисања који би резултовао непотпуним или некохерентним стањем диплома у бази.

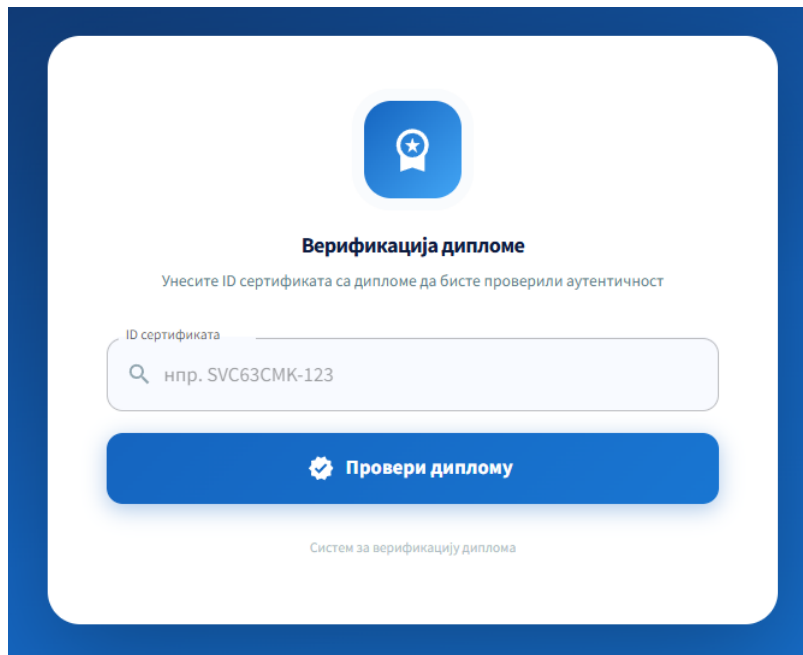
Листа диплома приказана је применом MUI DataGrid компоненте груписане по школама, са колонама за назив школе (са иконицом), место, број генерисаних диплома (приказан као Chip), типове присутних диплома, преузимање Excel резултата по школи и преузимање ZIP архиве са PDF дипломама школе. Типови диплома приказују се као скраћеничке ознаке у боји – ZZ за Златну захвалницу (тип 1), PR за Диплому I ранга (тип 2), F за Диплому финалног ранга (тип 3) и Z за Захвалницу (тип 4); конкретне боје део су тренутног дизајна и немају функционални значај.

У хедеру странице налазе се два дугмета за масовно преузимање на нивоу целе одабране године: „Сви резултати“ преузима ZIP архиву са Excel датотекама резултата свих школа (action=downloadAllResults), а „Све дипломе“ архиву са свим PDF дипломама (action=downloadAllDiplomas). Оба дугмета онемогућена су ако за одабрану годину не постоје резултати односно дипломе, чиме се спречава преузимање празних архива.

Јавна верификација аутентичности. Јавна страница за верификацију доступна је без аутентификације путем руте /verify и дизајнирана са посебним визуелним идентитетом – центрираном формом за унос сертификационог кода (Слика 7).

Страница нуди поље за унос кода са тренутном валидацијом формата, верификацију притиском типке Enter или кликом на дугме, анимирани приказ резултата (зелена картица за важећу, црвена за неважећу диплому), приказ свих детаља (име ученика, разред, школа, година, тип дипломе, награда), прилагодљив приказ за школске дипломе без поља ученика и разреда, дугме за верификацију друге дипломе без поновног учитавања странице.

Страница укључује помоћну функцију getDiplomaTypeLabel(), која нумеричку вредност template_type преводи у читљив назив применом i18next система, уз резервни приказ за непознате типове.



Слика 7: Јавна страница за верификацију аутентичности дипломе (/verify)

Школски преглед диплома. Страница намењена школским налозима омогућава самостални преглед и преузимање сопствених диплома без потребе за контактирањем администратора (Слика 8). Школски ID екстрахује се из JWT токена применом библиотеке jwtDecode и прослеђује свим API захтевима, а школа приступа искључиво дипломама својих ученика, што је имплементирано на серверском нивоу филтрирањем по school_id. Страница омогућава претрагу диплома по разреду и такмичарској години, преузимање индивидуалне дипломе ученика у PDF формату, преузимање Златне захвалнице и Захвалнице школе за одабрану годину, као и преузимање ZIP архиве са свим дипломама школе за годину.

Добродошли ПРИВАТНА ОСНОВНА ШКОЛА „МИХАЈЛО ПУПИН”
Тренутно активна фаза: **Крајњи резултати**
Српски (Ћирилица)

Дипломе
Преузмите дипломе ученика ваше школе
Година
2026
Златна захвалница (ZZ)
Захвалница (Z)
Преузми све дипломе

Претрага по имену
Одељење
Све
Награда
Све

#	Име и презиме	Одељење	Тип дипломе	Укупни п...	Награда	Диплома
1	Ленка Николић	I OS	PR	90	I награда	PDF
2	Трифун Антонијевић	I OS	PR	90	I награда	PDF
3	Сава Мандић	I OS	PR	85	II награда	PDF
4	Вукан Савић	I OS	PR	78,75	II награда	PDF
5	Георгије Стејић	I OS	F	77,5	III награда	PDF
6	Георгије Стејић	I OS	PR	77,5	II награда	PDF

Слика 8: Школска страница за преглед и преузимање диплома (SchoolDiplomasPage)

3.8. Клијентски сервис

Датотека `src/api/diplomaService.js` представља централни клијентски сервис за управљање дипломама. Организована је као колекција извезених функција (без класе), пратећи функционалну парадигму, и одговорна је за апстраховање свих HTTP позива ка API-ју, управљање животним циклусом Web Worker инстанци и конверзију резултата у PDF формат.

Сервис обједињује неколико кључних функција. Функција `getTemplateImageUrl(templateId)` конструише URL за преузимање слике шаблона са сервера применом константе `API_BASE_URL` и `id`-ја шаблона као `query` параметра.

Функција `loadImage(src)` враћа `Promise` са `HTMLImageElement` и поставља `crossOrigin` = `anonymous` пре `src` атрибута, чиме омогућава CORS подршку неопходну за цртање `cross-origin` слика на платну (без тога би покушај цртања „контаминирао“ платно и онемогућио позив `toBlob()`).

Функција `createWorkerStudentRecord(student, award, ...)` конструише стандардизовани запис ученика за Worker – обрађује различите формате назива поља (ћириличне и латиничне варијанте из различитих Excel формата), нормализује вредности и додаје интерне метаподатке (`diplomaId`, `templateType`, `certId`), што је кључно због разноликости извора Excel датотека.

Функција `runWorkerForSingleStudent()` покреће нову инстанцу Web Worker-а за генерисање дипломе једног ученика: преузима слику шаблона применом `loadImage()`, конвертује је у `ImageBitmap`, прослеђује Worker-у уз `Transferable Objects` и враћа `ArrayBuffer` резултат. Ова функција примарно се користи при регенерацији постојећих диплома – када се ажурирају подаци ученика (име, презиме, школа, матична школа, разред), регенерација осигурава да новогенерисана диплома одражава тренутне податке из базе, а не оне из тренутка првобитног генерисања, при чему је подржана за обе врсте такмичарских диплома.

Функција `createPdfBlobFromResult(result)` конвертује `ArrayBuffer` сликовних података у `jsPDF Blob`, аутоматски одређује оријентацију поређењем димензија слике са пропорцијом A4 формата и позиционира слику `letterbox` алгоритмом.

Коначно, функција `regenerateSingleDiploma(studentId)` представља комплетну регенерацију дипломе ученика – прво преузима метаподатке регенерације са сервера, подржава ученике са више диплома (нпр. и такмичарску диплому и захвалницу) кроз итерацију листе записа, за сваку диплому генерише нови PDF и отпрема га применом `uploadPdf()` са `regenerate=1`, враћајући последњи успешан резултат отпремања.

Логика одређивања оријентације и `letterbox` позиционирања, која је заједничка за све типове диплома, поједностављено изгледа овако:

```
function createPdfBlobFromResult(width, height, imgData) {
  const ratio = width / height;
  const orientation = ratio > 1 / Math.sqrt(2) ? 'landscape' : 'portrait';
  const doc = new jsPDF({ orientation, unit: 'pt', format: 'a4' });
  const pw = doc.internal.pageSize.getWidth();
  const ph = doc.internal.pageSize.getHeight();
```

```

const scale = Math.min(pw / width, ph / height);
const w = width * scale, h = height * scale;
const x = (pw - w) / 2, y = (ph - h) / 2; // центрирање
doc.drawImage(imgData, 'JPEG', x, y, w, h);
return doc.output('blob');
}

```

Сервис имплементира обраду грешака на више нивоа. На нивоу слике шаблона, ако се слика не може учитати (HTTP грешка или CORS проблем), Worker се ипак покреће са null bitmap-ом и у том случају генерише placeholder дипломе са сивом позадином и напоменом да слика шаблона није доступна. На нивоу отпремања, пре парсирања JSON одговора сервера систем проверава заглавље Content-Type – ако сервер врати одговор који није JSON (нпр. PHP грешка са echo нуспроизводима), систем бележи цео текст одговора у конзолу и баца описну грешку уместо тихог неуспеха при JSON.parse().

3.9. Web Worker за рендеровање

Web Worker представља кључну техничку компоненту система за рендеровање диплома. Извршава се у засебној нити оперативног система (off-main-thread), чиме спречава блокирање UI нити при рачунарски интензивним операцијама цртања, а датотека се учитава применом webpack-подржане синтаксе new URL('./workers/diplomaWorker.js', import.meta.url). Worker прима поруке преко postMessage() API-ја са структуром приказаном у Табели 14.

Табела 14: Структура поруке коју Web Worker прима преко postMessage().

Поље	Тип	Опис
id	string	Идентификатор захтева за руковање вишеструким Worker-има
students	array	Низ нормализованих записа ученика из createWorkerStudentRecord()
templateBitmap	ImageBitmap null	Bitmap шаблонске слике (или null ако учитавање није успело)
templateWidth	number	Ширина шаблонске слике у пикселима
templateHeight	number	Висина шаблонске слике у пикселима

Улазна тачка Worker-а је обрађивач onmessage, који прима поруку, за сваког ученика рендерује платно и враћа резултате уз листу преносивих објеката:

```

self.onmessage = async (e) => {
  const { id, students, templateBitmap, templateWidth, templateHeight } = e.data;
  const results = [];
  for (const student of students) {
    const canvas = new OffscreenCanvas(templateWidth, templateHeight);
    const ctx = canvas.getContext('2d');
    if (templateBitmap) ctx.drawImage(templateBitmap, 0, 0);
  }
}

```

```

renderDiploma(ctx, student, templateWidth, templateHeight);
const blob = await canvas.convertToBlob({ type: 'image/jpeg', quality: 0.7 });
const buffer = await blob.arrayBuffer();
results.push({ diplomaId: student.diplomaId, buffer });
}
self.postMessage({ id, results }, results.map((r) => r.buffer));
};

```

Срж рендеровања текста представља функција `drawWrappedText()`, која имплементира аутоматско преламање линија на основу максималне ширине (75% ширине платна или прилагодљив параметар), двоструко рендеровање (прво `strokeText()` белом бојом, затим `fillText()` тамном бојом ради читљивости на свим позадинама), вертикално позиционирање са конфигурабилним размаком међу линијама и централно хоризонтално поравнање (`textAlign = 'center'`), уз враћање Y координате последње нацртане линије ради правилног позиционирања следећег елемента. Њена поједностављена имплементација изгледа овако:

```

function drawWrappedText(ctx, text, x, y, maxWidth, lineHeight) {
  const words = text.split(' ');
  let line = '';
  for (const word of words) {
    const test = line ? line + ' ' + word : word;
    if (ctx.measureText(test).width > maxWidth && line) {
      ctx.strokeText(line, x, y); ctx.fillText(line, x, y);
      line = word; y += lineHeight;
    } else { line = test; }
  }
  ctx.strokeText(line, x, y); ctx.fillText(line, x, y);
  return y;
}

```

Логика рендеровања разликује се по типу дипломе. За Златну захвалницу (тип 1) систем приказује назив школе и место преузете из поља `excel_school_name` и `excel_school_location` која је администратор унео путем Excel датотеке – ово је намерно решење, јер Златна захвалница представља признање школи као институцији, па се користи тачан назив онако како је наведен у датотеци.

За Захвалницу (тип 4) назив школе и место читају се из поља матичне школе (`maticna_skola`) ученика у бази, чиме се на дипломи приказује матична школа уместо школе која га је пријавила за такмичење, што је важно када су то различите установе. Препоручује се да назив матичне школе буде уписан са зарезом на крају (нпр. „ОШ Душан Радовић,“) јер се назив школе и место приказују у два одвојена реда, а зарез их визуелно повезује у складну целину; ако ученик нема уписану матичну школу, систем прелази на назив школе која га је пријавила заједно са местом, чиме се осигурава да диплома увек садржи валидне податке.

За оба типа захвалница, испод основних података увек се приказује сертификациони код и верификациони URL малим фонтом (18–24px).

За такмичарске дипломе (тип 2 и 3) рендеровани садржај укључује пуно име ученика применом bold 126px Cambria фонта, разред применом 62px Cambria (вредност разреда прослеђује се из поља Разред учитаног из Excel датотеке, односно из колоне class табеле student путем JOIN операције при регенерацији, будући да се разред не чува у табели diplomas), назив матичне или тренутне школе са местом применом 62px Cambria, те назив награде применом bold 106px Cambria. За ученике виших разреда (вредности 9–12) врши се аутоматска конверзија у приказ од 1. до 4. разреда средње школе. Систем разликује случај Похвале, за коју се користи посебна текстуална формулација (остварио/ла запажени резултат), у односу на I, II и III награду, где се приказује одговарајући текст.

Worker враћа резултате применом Transferable Objects механизма ради максималне ефикасности. Сваки елемент резултата садржи ArrayBuffer са бинарним подацима слике, који се преносе (не копирају) назад у главну нит:

```
const transferList = results.map((item) => item.buffer);
self.postMessage({ id, results }, transferList);
```

Ова техника елиминише дупликацију меморије: при генерисању 500 диплома резолуције 1024×768, трансфер уместо копирања уштеди приближно 150–300 МБ меморије, што директно утиче на стабилност апликације при раду са великим серијама.

Пример визуелног изгледа генерисане такмичарске дипломе, са распоредом имена ученика, разреда, школе, награде, сертификационог кода и верификационог URL–а, приказан је на Слици 9.



Слика 9: Пример генерисане такмичарске дипломе (тип 2) са персонализованим подацима и сертификационим кодом у дну.

4. Систем верификације диплома

Систем верификације диплома обезбеђује криптографски јединствену идентификацију сваке издате дипломе применом случајно генерисаних сертификационих кодова и омогућава свим корисницима да верификују аутентичност дипломе путем јавне веб странице, без потребе за налогом на платформи. Тиме се решава један од кључних недостатака ранијег, ручног процеса – немогућност поузданог утврђивања да ли је конкретна диплома заиста издата од стране организатора такмичења.

Концепт сертификационог кода ослања се на једноставну, али моћну идеју: сваки документ носи насумичан, тешко погодив идентификатор који је трајно повезан са записом у бази података. За разлику од визуелних елемената заштите попут печата или потписа, који се на дигиталном документу лако копирају, овакав код омогућава да се аутентичност потврди упоредном провером са званичном евиденцијом, а не само визуелним прегледом. Управо због тога је верификација имплементирана као засебан, јавно доступан механизам, независан од остатка административног дела платформе.

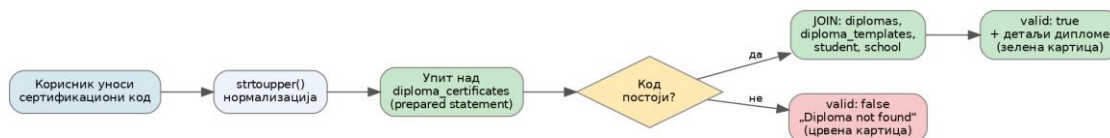
Сертификациони код генерише метода `generateCertificationId()` у класи `DiplomaVerifyController`. Код се састоји из два дела раздвојена цртицом: осам насумично изабраних знакова из скупа `A–Z` и `0–9` применом PHP функције `random_int()` (криптографски сигурна генерација, за разлику од `rand()`), и нумерички ID дипломе из базе података. Пример генерисаног кода је `AB3X7Q2N–42`. Будући да се сваки од првих осам карактера бира из скупа од 36 могућих вредности (26 великих слова и 10 цифара), укупан број комбинација износи $36^8 = 2.821.109.907.456$ (приближно 2,82 билиона) различитих кодова, што практично искључује могућност погађања важећег кода.

Поједностављен приказ генерисања кода у PHP–у изгледа овако:

```
public function generateCertificationId(int $diplomaId): string {
    $chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
    $code = '';
    for ($i = 0; $i < 8; $i++) {
        $code .= $chars[random_int(0, strlen($chars) - 1)];
    }
    return $code . '-' . $diplomaId;
}
```

Корисник уноси сертификациони код на верификационој страни, након чега систем шаље GET захтев на `api/diplomasVerify.php?id={certId}`. Контролер нормализује код у велика слова применом `strtoupper()` (код је за кориснике неосетљив на величину слова), претражује табелу `diploma_certificates` применом припремљеног упита и, у случају налаза, извршава вишеструки JOIN преко табела `diploma_certificates`, `diplomas`, `diploma_templates`, `student` и `school`. Тиме се све информације неопходне за потпун приказ верификационог резултата добијају у једном упиту, чиме се избегава N+1 проблем; резултат укључује `type_label` (превод нумеричке вредности типа у

читљив назив), is_school_level (за условни приказ поља ученика и разреда), као и is_zlatna и is_zahvalnica за специфичне приказе. Цео ток верификације, од уноса кода до приказа резултата, приказан је на Слици 10.



Слика 10: Ток процеса верификације дипломе – од уноса сертификационог кода до приказа резултата.

Срж SQL упита који враћа све потребне податке у једном пролазу, поједностављено, изгледа овако:

```

SELECT dc.certification_id, d.year AS diploma_year,
       d.award, dt.type AS template_type,
       s.name AS student_name, s.class,
       sc.name AS school_name
FROM diploma_certificates dc
JOIN diplomas d          ON d.id = dc.diploma_id
LEFT JOIN diploma_templates dt ON dt.id = d.template_id
LEFT JOIN student s      ON s.id = dc.student_id
LEFT JOIN school sc      ON sc.id = dc.school_id
WHERE dc.certification_id = :cid;
  
```

Метода createCertificate() у контролеру DiplomaVerifyController обезбеђује очување постојећих сертификата при регенерацији диплома. Уколико за дати diploma_id већ постоји сертификат у бази (случај регенерације), метода ажурира поља student_id и school_id, али задржава постојећу вредност certification_id без измене. На овај начин верификациони кодови остају непромењени кроз поновна генерисања, па корисник може успешно верификовати диплому чак и након обнове PDF документа или промене шаблона. Ова особина је од суштинског значаја јер би у супротном раније подељене дипломе постале неважеће после сваке регенерације.

Свака генерисана диплома садржи два текстуална поља у доњем делу: сертификациони код у формату „БРОЈ СЕРТИФИКАТА: {certId}“ и верификациони URL у формату „За верификацију посетите: {BASE_URL}/verify“. Ова поља се аутоматски наносе у Worker-у при рендеровању, малим фонтом (18–24px) са белим контурним ефектом, тако да су читљива а не нарушавају визуелни идентитет дипломе.

Овакав дизајн система верификације остварује две важне особине истовремено. Прво, поузданост – будући да се сертификат генерише криптографски сигурном функцијом и чува у засебној табели са ограничењем јединствености, практично је немогуће фалсификовати важећи код или га случајно поновити.

Друго, једноставност за крајњег корисника – верификација не захтева налог нити инсталацију додатних алата, већ само унос кода на јавно доступној страници, што је кључно јер примаоци диплома (родитељи, школе, будуће институције) најчешће нису корисници платформе. На тај начин дигитална диплома добија исту, или чак већу, тежину као физички документ са печатом.

5. Интернационализација и додатне измене

Нови модул потпуно је интегрисан са постојећим `i18next` локализационим системом платформе. Сви текстуални садржаји у интерфејсу (наслови, поруке, ознаке поља, поруке грешака) решавани су применом функције `t()` из `hook-a-useTranslation()`. За потребе система диплома додати су простори имена (`namespace` кључеви) приказани у Табели 15, при чему кључеви подржавају интерполацију за динамичке вредности – на пример, `diplomaVerifyPage.verifiedId` прима параметар `{id}`: „Сертификациони код: `{id}`“.

Табела 15: Простори имена (`namespace` кључеви) додати за систем диплома.

Кључ	Употреба
<code>diplomaTemplate.*</code>	Сцена управљања шаблонима (наслови, типови, дугмад, поруке)
<code>diplomasBySchool.*</code>	Административна сцена диплома по школи
<code>diplomaVerifyPage.*</code>	Јавна страница верификације
<code>schoolDiplomasPage.*</code>	Школска страница диплома

Функционалност претраге у сценама `DiplomaTemplates` и `DiplomasBySchool` користи помоћне функције `cyrillicToLatin()` и `latinToCyrillic()` из постојећег модула `src/helper/helperFunction`. Ова конверзија омогућава претрагу назива написаних ћирилицом уносом латиничких знакова и обрнуто, што је критична функционалност јер називи школа и ученика у систему могу бити у мешовитом писму.

Без овакве конверзије, корисник би морао тачно да погоди писмо у коме је назив уписан, што би значајно отежало проналажење жељених записа. Ова особина је нарочито важна у домаћем контексту, где се исти називи школа и имена ученика у пракси појављују и ћирилицом и латиницом, у зависности од извора података и навика особе која их уноси.

У оквиру истог циклуса извршене су и измене у клијентском рутирању и навигацији. У датотеци `App.js` додате су нове `React Router` руте за модул диплома: `/diploma-templates` (управљање шаблонима, само администратори), `/diplomas-by-school` (административни преглед диплома, само администратори) и `/school-diplomas` (школска страница за преузимање) – све заштићене компонентом `AdminRoute` односно одговарајућом провером улоге – као и јавна рута `/verify` за верификацију аутентичности доступна свим корисницима.

У компоненти `AdminSidebarItems.js` додате су нове ставке менија у административни сајдбар, у одељку „Управљање дипломама“: Шаблони диплома, Дипломе по школама и Верификација дипломе; школски сајдбар (`SchoolSidebarItems.js`) проширен је истоименим одељком са ставкама Дипломе и Верификација дипломе.

6. Обрада грешака и стабилност система

Систем за управљање дипломама имплементира вишеслојну стратегију обраде грешака која обухвата клијентски JavaScript, серверски PHP и комуникациони слој између нити. Циљ ове стратегије јесте да појединачни проблем – попут неисправне слике или привремене грешке на серверу – не доведе до пада читаве операције генерисања.

Полазна претпоставка при пројектовању обраде грешака била је да ће се, при раду са великим бројем спољних података и датотека, грешке неизбежно дешавати – било због неисправног уноса, привремених мрежних проблема или оштећених датотека. Уместо да се такве ситуације третирају као изузеци који заустављају рад, систем их третира као очекивано стање и настоји да настави рад са што мањим губитком. Овакав приступ, познат као отпорност на грешке (енг. *fault tolerance*), посебно је важан код дуготрајних операција као што је масовно генерисање, где би прекид на пола посла значио велики губитак времена.

На клијентској страни, процес генерисања умотан је у `try/catch` блокове на нивоу сваке појединачне дипломе. Ако генерисање одређене дипломе не успе, систем бележи грешку у конзолу са детаљним контекстом (име ученика, ID дипломе) и наставља са следећом, уместо да прекида целу серију; на крају се приказује `Snackbar` са бројем успешно и неуспешно генерисаних диплома. Пре парсирања JSON одговора систем проверава заглавље `Content-Type`, па ако сервер врати одговор који није JSON (нпр. PHP упозорење са `echo` нуспроизводима), бележи цео текст одговора у конзолу и баца описну грешку, чиме се спречавају тихе грешке при `JSON.parse()` позиву на невалидан улаз. Ако читавање слике шаблона не успе, `Worker` не прекида генерисање – уместо шаблона генерише `placeholder` дипломе са сивом позадином, оквиром, текстом „DIPLOMA“ и напоменом, при чему се сви персонализовани подаци и даље успешно наносе.

На серверској страни сви PHP контролери умотавају логику у `try/catch` блокове са `Exception handler`-ом. Грешке се бележе применом функције `error_log()` и враћају клијенту у JSON формату (`{ "error": "описна порука" }`), уз постављање HTTP статуса 400 за грешке валидације или 500 за серверске грешке. `DiplomasController` садржи и посебне методе за одржавање интегритета система: `cleanupMissingFiles()` проналази записе са `pdf_path` који указује на непостојеће датотеке и поставља `pdf_path` на `NULL`, док `cleanupOrphanedFiles()` проналази физичке датотеке у `uploads` директоријуму без записа у бази и брише их. Ове методе омогућавају да база и датотечни систем временом остану међусобно усклађени.

Грешке у `Worker` нити хватају се применом обрађивача `worker.onerror` на нивоу главне нити. Сваки позив `Worker`-а умотан је у `Promise` са `reject` обрађивачем за истек времена и неочекиване грешке, а `Worker` се зауставља применом `worker.terminate()` после завршетка или грешке ради ослобађања ресурса; без `terminate()`, `Worker` инстанца наставила би да постоји у меморији и непотребно трошила ресурсе.

7. Безбедност система

Безбедност новог модула обезбеђена је на више нивоа – кроз заштиту од SQL injection напада, заштиту путања датотека и строгу ауторизацију приступа дипломама. С обзиром на то да систем барата личним подацима ученика и званичним документима, безбедносним аспектима посвећена је посебна пажња.

Безбедносни модел система заснован је на начелу најмањих привилегија и на претпоставци да клијентској страни не треба веровати. Свака осетљива операција пролази кроз проверу на серверу, а подаци се филтрирају према идентитету корисника утврђеном из JWT токена, а не према вредностима које клијент сам шаље. У наставку су детаљније описане три главне линије одбране примењене у новом модулу.

Сви SQL упити у оквиру новог модула извршавају се применом припремљених упита (prepared statements) и параметаризованих вредности. Класа DB.php обезбеђује да се кориснички унос никада не уграђује директно у SQL наредбе, већ се вредности прослеђују као параметри у облику асоцијативних низова са именованим ознакама (нпр. :param_name), чиме се постиже виши ниво безбедности и заштита од SQL injection напада. На овај начин злонамерни унос се третира искључиво као податак, а никада као део SQL синтаксе.

При приступу датотекама на серверу систем користи функцију realpath() за резолуцију апсолутне путање, а затим проверава постојање резолвиране датотеке применом file_exists(). Овим се спречавају path traversal напади у сценаријима у којима би злонамерни корисник покушао да приступи датотекама ван дозвољеног директоријума, на пример уношењем низова попут „../“ у путању.

Контролер SchoolDiplomasController обезбеђује да школски налози могу приступати искључиво сопственим дипломама применом клаузуле WHERE d.school_id = :school_id у свим SQL упитима. Верификација school_id-ја врши се на серверском нивоу, на основу вредности из JWT токена, тако да клијентски код не може заобићи ову проверу ни манипулацијом HTTP захтева. Тиме је онемогућено да једна школа, чак и намерним мењањем захтева, приступи дипломама друге школе.

Важно начело примењено у читавом модулу јесте да се ниједна безбедносно осетљива одлука не доноси на клијентској страни. Клијентски код служи искључиво за приказ и удобност корисника, док се све провере приступа, валидације и ауторизације спроводе на серверу, где их корисник не може заобићи. Овакав приступ, познат као одбрана по дубини, значи да чак и у случају да нападач у потпуности контролише клијентску апликацију, не може приступити подацима нити операцијама за које нема одговарајућа права на серверу.

8. Перформансе и оптимизације

Систем за генерисање диплома суочава се са изазовом обраде потенцијално стотина или хиљада ученика у једном увозу, па је примењено неколико оптимизационих техника за обезбеђење прихватљивих перформанси без деградације корисничког искуства. Оптимизације су усмерене како на брзину обраде, тако и на одзивност интерфејса и потрошњу меморије.

Кључна оптимизација је off-main-thread обрада применом Web Worker-a. Тестови показују да генерисање 500 диплома траје два до три минута на просечном рачунару; без Web Worker-a UI би био потпуно блокиран током целог периода, онемогућавајући кориснику праћење напретка или вршење других операција, док уз Worker корисничко окружење остаје потпуно функционално. Уз то, трансфер ArrayBuffer-a применом Transferable Objects (трећи параметар postMessage()) елиминира дуплирање меморије при комуникацији између нити – власништво над меморијским блоком преноси се у Worker, а по завршетку назад у главну нит без копирања, што при генерисању 500 диплома резолуције 1024×768 уштеди приближно 150–300 МБ меморије.

Поред тога, у сценама за генерисање диплома серијска обрада уметнуте yield тачке применом `await new Promise(resolve => setTimeout(resolve, 0))` после сваких N диплома, чиме се омогућава да React рендерује ажурирани прогрес индикатор између итерација и спречава привидно замрзавање интерфејса.

Листе диплома у компонентама DiplomasBySchool и SchoolDiplomasPage учитавају само релевантне записе применом SQL клаузула WHERE, ORDER BY и LIMIT, а MUI DataGrid управља серверском пагинацијом са конфигурабилном величином странице, што је критично јер систем може садржати хиљаде диплома за вишегодишње такмичење.

Коначно, масовно преузимање диплома имплементира се серверским генерисањем ZIP архива применом PHP класе ZipArchive, што је знатно ефикасније од клијентског генерисања: избегава трансфер великих количина бинарних података преко мреже, ZIP компресија у PHP-у је уграђена и брза, а клијент добија јединствену URL адресу за преузимање уместо стотина засебних захтева.

9. Тестирање

Модул за управљање дипломама тестиран је кроз неколико нивоа верификације функционалности, покривајући све главне корисничке токове и граничне случајеве.

Тестирање је организовано у функционално тестирање појединачних целина и проверу понашања система у граничним случајевима, са циљем да се потврди исправан рад како у уобичајеним, тако и у неочекиваним ситуацијама.

У оквиру функционалног тестирања шаблона диплома проверени су следећи токови: креирање шаблона са учитавањем PNG слике и провера складиштења у директоријуму `uploads/diploma_templates/`; измена шаблона са заменом слике и провера брисања старе датотеке са диска; брисање шаблона уз потврду и провера физичког брисања датотеке; претрага шаблона ћириличним уносом и провера `latinToCyrillic` конверзије; те филтрирање по типу и такмичарској години.

Тестирање генерисања диплома обухватило је увоз Excel датотеке са листом од преко 100 ученика и проверу парсирања свих колона, генерисање диплома сва четири типа уз проверу различитих рендеровања у Worker-у, верификацију да сваки PDF садржи тачне персонализоване податке, проверу присуства сертификационог кода и верификационог URL-а на свакој дипломи, као и регенерацију постојеће дипломе уз проверу ажурирања `pdf_path` и задржавања вредности `certification_id`.

Тестирање верификације аутентичности обухватило је верификацију важећег сертификационог кода уз проверу тачности свих приказаних података, верификацију неважећег кода уз проверу приказа поруке о грешци одговарајуће боје, верификацију школске дипломе уз проверу да поља ученика и разреда нису приказана, те верификацију уноса малим словима уз проверу `strtoupper()` нормализације.

Поред главних токова, посебно су тестирани гранични случајеви који лако измичу пажњи током развоја: ученик без матичне школе у бази (систем враћа назив школе из Excel датотеке), ученик са разредом већим или једнаким 9 (провера приказа коригованог броја разреда, нпр. 9 → 1. средња), учитавање Excel датотеке са годином различитом од шаблона (приказ упозорења), учитавање датотеке са празним именима ученика (прескакање генерисања за тај ред), непостојећа слика шаблона са HTTP 404 одговором (генерисање placeholder дипломе), као и регенерација ученика са више диплома. Провером ових случајева потврђено је да систем остаје стабилан и предвидив и у неуобичајеним условима.

Тестирање је спроведено комбинацијом ручне провере кроз кориснички интерфејс и непосредне провере стања у бази података и систему датотека након сваке операције. За сценарије масовног генерисања коришћени су тестни Excel скупови различите величине, од неколико десетина до неколико стотина ученика, како би се потврдило да се перформансе и потрошња меморије понашају очекивано при порасту обима. Посебна пажња посвећена је идемпотентности регенерације – вишеструко покретање регенерације исте дипломе не сме да доведе до дуплирања записа, промене сертификационог кода нити нагомилавања застарелих PDF датотека на диску.

10. Одржавање и унапређење система

Поред тренутне функционалности, систем оставља простор за више праваца даљег развоја. Тренутна верзија захтева ручно покретање генерисања диплома од стране администратора, па би у будућим верзијама било могуће увести механизам аутоматског генерисања након финализације резултата такмичења, чиме би се елиминисала потреба за ручним покретањем и смањила могућност људске грешке.

Постојећи систем верификације, заснован на сертификационим кодовима, могао би се проширити генерисањем QR кода за сваку диплому који би садржао линк ка страници за верификацију, тако да корисник скенирањем директно приступа информацијама о валидности дипломе без ручног уноса кода.

Будући развој може обухватити и напредније механизме претраге и филтрирања диплома, као и аналитичке извештаје о броју издатих диплома, најуспешнијим школама и трендовима учешћа.

Иако тренутно решење обезбеђује чување архивираних диплома, могуће је развити посебан административни интерфејс за преглед архиве, претрагу архивираних записа и враћање диплома у активно стање, чиме би се додатно повећала поузданост система и олакшало управљање историјским подацима.

Са становишта одрживости кода, доследна примена MVC обрасца на серверу и компонентне архитектуре на клијенту чини да се сва наведена проширења могу увести постепено, уз минималан ризик по постојеће функционалности. Будући да је нови модул потпуно адитиван и не дира у постојеће endpoints, његово даље развијање може тећи паралелно са редовним радом платформе. Препоручује се и увођење аутоматизованих тестова за критичне токове – генерисање, регенерацију и верификацију – чиме би се олакшало одржавање и смањила могућност регресије приликом будућих измена.

11. Закључак

Систем за генерисање и валидацију дигиталних диплома представља комплексан, вишеслојни модул чија је имплементација захтевала интеграцију бројних технологија и пажљиво пројектовање архитектуре. Применом Web Workers технологије за off-main-thread рендеровање, jsPDF библиотеке за PDF генерисање, криптографски јединственог сертификационог система за верификацију аутентичности, те потпуне MVC архитектуре на серверу, постигнута је потпуна дигитализација процеса издавања диплома у оквиру платформе „Кенгур без граница“.

Компаративном анализом старе и нове верзије репозиторијума утврђено је да нови модул уводи тачно 9 нових фронтенд компоненти и 12 нових бекенд датотека (4 endpoints, 4 контролера и 4 модела), уз 4 нове MySQL табеле (diplomas, diploma_templates, diploma_certificates и archived_school_diplomas) и измену постојеће табеле student додавањем колоне maticna_skola. Имплементација је потпуно адитивне природе, што значи да ниједна постојећа функционалност платформе није нарушена увођењем новог модула.

Будући рад на систему може укључити додавање email нотификација за школе при доступности нових диплома, те интеграцију QR кода на дипломама за директно скенирање и верификацију мобилним уређајем. Имајући у виду модуларну архитектуру система, оваква проширења могуће је увести постепено, без већих измена постојећег кода.

Литература

- [1] React – A JavaScript library for building user interfaces. <https://reactjs.org/> (Последњи приступ: Мај 2026)
- [2] Material UI – React component library implementing Google Material Design. <https://mui.com/> (Последњи приступ: Мај 2026)
- [3] jsPDF – Client-side JavaScript PDF generation. <https://github.com/parallax/jsPDF> (Последњи приступ: Јун 2026)
- [4] MDN Web Docs: Web Workers API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API (Последњи приступ: Јун 2026)
- [5] MDN Web Docs: OffscreenCanvas API. <https://developer.mozilla.org/en-US/docs/Web/API/OffscreenCanvas> (Последњи приступ: Јун 2026)
- [6] SheetJS Community Edition. <https://sheetjs.com/> (Последњи приступ: Мај 2026)
- [7] PHP Documentation: ZipArchive. <https://www.php.net/manual/en/class.ziparchive.php> (Последњи приступ: Мај 2026)
- [8] PHP Documentation: random_int(). <https://www.php.net/manual/en/function.random-int.php> (Последњи приступ: Мај 2026)
- [9] PHP Data Objects (PDO). <https://www.php.net/manual/en/book.pdo.php> (Последњи приступ: Мај 2026)
- [10] i18next Documentation. <https://www.i18next.com/> (Последњи приступ: Мај 2026)
- [11] React Router v6 Documentation. <https://reactrouter.com/> (Последњи приступ: Мај 2026)
- [12] MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/> (Последњи приступ: Јун 2026)
- [13] file-saver npm package. <https://github.com/eligrey/FileSaver.js/> (Последњи приступ: Мај 2026)
- [14] Kangaroo Without Borders Official Website. <https://www.mathkang.org/> (Последњи приступ: Мај 2026)

Кратка биографија кандидата

Мирко Михајловић рођен је 2. јануара 2003. године у Новом Пазару. Основно образовање стекао је у Основној школи „Јосиф Панчић“ у Брвенику, коју је завршио 2017. године. Средњу школу похађао је у Гимназији Рашка, математички смер, коју је завршио 2021. године.

Исте године уписао је студије информатике на Природно–математичком факултету у Крагујевцу. Током студирања развио је страст према програмирању, која је постепено преузела водећу улогу у односу на математику, којој је до тада давао предност.

У јуну 2025. године засновао је радни однос у компанији НТЕС, где ради на позицији frontend софтверског инжењера. На тој позицији активно ради и данас.

Универзитет у Крагујевцу
Природно-математички факултет
Институт за математику и информатику

Завршни рад под називом

**ГЕНЕРИСАЊЕ И ВАЛИДАЦИЈА ДИГИТАЛНИХ ДИПЛОМА У ОКВИРУ
ПЛАТФОРМЕ „КЕНГУР БЕЗ ГРАНИЦА“**

одбрањен је _____.

МЕНТОР:

др Марина Свичевић,
ванредни професор, ПМФ Крагујевац

ЧЛАНОВИ КОМИСИЈЕ:

др Татјана Стојановић,
доцент, ПМФ Крагујевац

др Александар Миленковић,
доцент, ПМФ Крагујевац

Завршни рад је оцењен оценом _____.